Kế hoạch thi hành — tầng A

thứ tự làm, cổng quyết định, ngân sách

Mục lục

1 Đã chốt **2**

2 Nguyên tắc xếp thứ tự **3**

3 Giai đoạn 0 — Hạ tầng · 0 lượt LLM **3**

3.1 A Lấy hiện vật và kiểm kê . . . 3

3.2 B Đo cách cắt chunk — không phải quy định nó . . . 4

3.3 C Pool — BM25 và bốc ngẫu nhiên trên ĐÚNG tập node document . . . 4

3.4 D Rerank theo từng nguồn . . . 4

3.5 E Khung judge . . . 4

3.6 F Bộ chẻ câu hỏi . . . 5

3.7 G Bộ NLI và bảng phủ . . . 5

3.8 H Đo lường theo từng bước . . . 5

3.9 J Chia tập — việc MỚI, do hai quyết định sinh ra . . . 5

4 Giai đoạn 0.5 — Chạy thử cả bốn cổng trên test index · $945 **6**

4.1 CỔNG 0.5 — có đáng index train không . . . 7

5 Giai đoạn 1 — Hiệu chuẩn chuẩn ngoài · $185 **7**

5.1 CỔNG 1 — `adq` thành gì . . . 8

6 Giai đoạn 2 — Pha 1, đào nhãn · ~$3.850 **8**

7 Giai đoạn 3 — Surrogate · 0 lượt LLM **8**

7.1 CỔNG 2 — kiến trúc . . . 9

8 Giai đoạn 4 — Cổng `p` · miễn phí **9**

8.1 CỔNG 3 — có triển khai 200k không . . . 9

9 Giai đoạn 5 — Pha 2, sản xuất theo lô · ~$8.500 **10**

9.1 CỔNG 4 — ở ranh giới MỖI lô . . . 10

10 Giai đoạn 6 — Tầng B 11

11 Ngân sách 11

12 Phải chốt trước khi giai đoạn 0 chạy 11

13 Rủi ro và dấu hiệu phát hiện 12

14 Những con số chưa ai đo 12

Thứ tự làm, cổng quyết định, ngân sách, và những gì phải chốt trước khi bắt đầu.

Thiết kế nằm ở tài liệu chính §3 và pipeline chi tiết. Tài liệu này chỉ nói **làm theo thứ tự nào, và dừng ở đâu nếu sai**.

Tầng B không nằm trong kế hoạch này — nó bắt đầu sau khi tầng A xuất được nhãn.

---

1. Đã chốt

```
retriever      G-reasoner, checkpoint PRETRAIN phát hành (rmanluo/G-reasoner-34M)
               KHÔNG chạy sft_training - tức không fine-tune trên dữ liệu của ta
corpus         CHÍNH tập train của G-reasoner:
                   277.839 truy vấn * 2.972.931 document
                   từ train set của HotpotQA * MuSiQue * 2Wiki
ngôn ngữ       tiếng Anh
chunk          một entry của documents.json = một document node
               = một đoạn Wikipedia có sẵn của dataset.  KHÔNG cắt lại.
```

Vì sao không chạy `sft_training`**.** Nếu fine-tune retriever trên nhãn ta đào, rồi lại dùng retriever đó dựng pool để đào nhãn, thì vòng lặp tự khép: pool đã mã hóa sẵn tín hiệu ta đang tìm, và nhãn sinh ra chỉ phục hồi thứ model đã biết. Ô «lỗi đồ thị» §6.1 sẽ không bao giờ khác rỗng. Bỏ bước fine-tune giữ pool **độc lập với tín hiệu đang đào**.

Cái giá: nhánh GFM (15/40) yếu hơn, nên tỷ lệ *"pool thiếu"* sẽ **cao hơn** con số `45%` suy từ paper — vì `HotpotQA 44,6%` / `MuSiQue 46,2%` đo với retriever bình thường. Ngân sách pha 2 vì thế là **lạc quan**, và `0.J/GĐ 2.1` phải đo lại.

Index: có sẵn cho TEST, không có cho TRAIN

```
CÓ SẴN     hotpotqa_test_v2 * musique_test * 2wikimultihopqa_test
           processed/stage1/ dựng sẵn, tải từ OneDrive của tác giả      ->  $0

KHÔNG CÓ   hotpotqa_train * musique_train * 2wikimultihopqa_train
           doc gfm-rag: "must be indexed locally using Stage 1 scripts"

CÓ SẴN     raw của CẢ HAI - train.json + supporting_documents miễn phí
```

Dựng index nghĩa là chạy **OpenIE bằng LLM trên từng document** (GFM-RAG §KG-index), nên chi phí tỷ lệ với số document:

```
~800 token vào + ~150 token ra mỗi document   ~  $0,0042/doc      (GIẢ ĐỊNH, 0.A đo lại)
10,7 document mỗi truy vấn                       (2.972.931 / 277.839)
```

```
phần pha 1   18.200 truy vấn  ->  ~195.000 doc    ->    ~$820
phần pha 2  200.000 truy vấn  ->  ~2,14 triệu doc ->  ~$9.000
```

`$9.800` **này không nằm trong bảng ngân sách cũ.** Nhưng nó là **quyết định sau cổng**, không phải chi phí đầu vào — xem `GĐ 0.5`.

---

2. Nguyên tắc xếp thứ tự

```
Bốn trong năm cổng là MIỄN PHÍ hoặc gần miễn phí,
và chúng nằm TRƯỚC 75% chi phí.
CỔNG 0.5 tốn $945 và chặn $13.000 - rẻ nhất, chặn nhiều nhất.
```

Mỗi giai đoạn phải trả lời được một câu hỏi *"có nên đi tiếp không"* **trước** khi giai đoạn sau tiêu tiền. Đây không phải cẩn thận hình thức: bảng chi phí `$23.300` treo vào năm con số chưa ai đo, và mỗi cổng đo đúng một trong số đó. Riêng `$9.820` indexing thì cổng `0.5` quyết có tiêu hay không — nó không phải chi phí đầu vào.

---

3. Giai đoạn 0 — Hạ tầng · 0 lượt LLM

Tốn công nhiều nhất, tốn tiền ít nhất. Không giai đoạn nào sau chạy được nếu chỗ này sai.

Hai quyết định ở mục *Đã chốt* xóa hẳn việc chọn cách cắt chunk, nhưng thêm việc **đo** nó và việc **chia tập**.

3.1 A Lấy hiện vật và kiểm kê

```
checkpoint     rmanluo/G-reasoner-34M        (còn rmanluo/GFM-RAG-8M là bản cũ hơn)
code           github.com/RManLuo/gfm-rag
đồ thị         processed/stage1/  gồm ba CSV:
                   nodes.csv      tên node * type thuộc {entity, document, summary} * attribute
                   relations.csv  tên quan hệ
                   edges.csv      (source, relation, target)
```

Lưu ý `stage1` ở đây là **đầu ra dựng đồ thị**, không phải chặng huấn luyện. Và `attribute` là **cột** trên node, không phải loại node riêng — nên ba loại node, không phải bốn.

Phải trả lời trước khi đi tiếp:

```
1  đồ thị đã dựng sẵn có được PHÁT HÀNH không, hay ta phải tự dựng?
   tự dựng trên 2.972.931 document là một job thật - phải định phạm vi
2  paper nói có BA constructor khác nhau  ->  dùng cái nào?  chốt MỘT.
3  MỘT đồ thị cho cả ba bộ, hay MỘT MỖI BỘ?
   truy vấn HotpotQA phải truy hồi trong corpus HotpotQA -
   trộn ba bộ là sai, pool sẽ đầy đoạn không liên quan từ bộ khác
4  đếm:  truy vấn mỗi bộ * bao nhiêu có supporting_documents
         * bao nhiêu có phân rã (MuSiQue)
```

3.2 B Đo cách cắt chunk — không phải quy định nó

```
documents.json = { "tên tài liệu": "nội dung", ... }
MỖI ENTRY LÀ MỘT DOCUMENT NODE.  Framework không cắt chunk bên trong.
```

Nên đơn vị chunk đã được dataset quyết: **một đoạn Wikipedia**. Việc còn lại là đo, vì mọi bảng chi phí dựa vào nó:

```
đọc node type == "document" từ nodes.csv, đo:
    phân bố độ dài token
    CÓ chồng lấn giữa các đoạn không?
        -> nếu CÓ thì phải khử trùng lặp trước khi lập pool,
           vì chunk chồng nhau tạo dư thừa nhân tạo - đúng thứ §3.6 sinh ra để phát hiện
```

`cost(S)  500 + 250·|S|` **phải tính lại theo độ dài THẬT.** Con số `250` là giả định; đoạn Wikipedia trung bình có thể dài hơn hẳn. Lệch ở đây thì mọi bảng chi phí trong cả ba tài liệu lệch theo.

3.3 C Pool — BM25 và bốc ngẫu nhiên trên ĐÚNG tập node document

```
pool ứng viên      = các node type == "document"
index BM25         = dựng trên ĐÚNG văn bản của các node đó
bộ bốc ngẫu nhiên  = bốc từ ĐÚNG tập đó
```

Khi đó tính đồng nhất cách cắt là **đúng theo cấu trúc**, không phải nhờ kỷ luật. Tiền kiểm §6.1 (*"đoạn lấy về không có node nào trong đồ thị"*) trở thành **không thể fail** — mọi đoạn trong pool đều là một node, vì nó lấy từ đồ thị ra.

Đây là chỗ hai quyết định của bạn xóa rủi ro số một của bản kế hoạch trước.

3.4 D Rerank theo từng nguồn

```
K <= 60 thô  ->  khử trùng lặp  ->  rerank THEO TỪNG NGUỒN
n = 40  =  20 BM25  +  15 GFM  +  5 ngẫu nhiên
```

Hạn ngạch riêng, **không gộp chung**: reranker chia sẻ điểm mù với retriever, gộp chung thì đoạn mà BM25 lôi về đúng *vì* GFM không thấy sẽ bị loại ngay.

Thêm một phép đo **miễn phí** khi đã có nhãn:

```
tỷ lệ đóng góp vào T theo TỪNG NHÁNH NGUỒN
->  nói cho ta biết hạn ngạch 20/15/5 đặt đúng chưa
->  và nó là cách duy nhất nhìn thấy rò rỉ ở 0.J
```

3.5 E Khung judge

```
sinh    â_S   =  generate( q, S )
chấm    v(S)  =  P_judge( "đúng" | q, â_S, y, aliases )    đọc logprob token ĐẦU
                 KHÔNG decode câu trả lời của judge
                 KHÔNG dùng thang 1-5 - 5 mức làm hàng chục tập con hòa nhau
```

```
                PHẢI nhét answer_aliases vào prompt: "Zemeckis" vs
                "Robert Zemeckis" bị chấm No thì v(S) thấp giả và tập
                ĐÚNG bị loại oan

cache   khóa = hash( model, template, nội dung chunk đã sắp )
               băm NỘI DUNG chứ không phải tên - chunk lại corpus là tên va nhau
gom lô  xếp chunk theo thứ tự cố định, gom lô theo tiền tố chung
            S_1 = {c1,c3,c7}
            S_2 = {c1,c3,c9}   chung tiền tố "c1 c3" -> chỉ encode đuôi
        vòng 1 greedy sinh đúng dạng đó (mọi ứng viên là S + {k} cùng S)
```

□ Nếu prompt caching **không ăn** thì `cost(S)` sai. Đo token thật ở đây, đừng tin công thức.

3.6 F Bộ chẻ câu hỏi

```
nhánh MuSiQue           DÙNG phân rã gán sẵn (câu hỏi 1-hop + đáp án trung gian)
                        0 lượt LLM
nhánh HotpotQA / 2Wiki  LLM, 1 lượt MỖI TRUY VẤN (không phải mỗi tập con), ~400 token
```

Và dùng nhánh MuSiQue để **đo** xem LLM chẻ có sót hop không — đối chiếu phân rã LLM sinh với phân rã đã gán. Đây là cách duy nhất kiểm bước này mà không cần người gán.

3.7 G Bộ NLI và bảng phủ

```
e(k, j)  =  NLI( premise = đoạn k,  hypothesis = mảnh j )
            thuộc { entail, neutral, contradict }
40 x J lượt forward model nhỏ, 0 lượt API
```

Corpus là **tiếng Anh**, nên chọn được model NLI sẵn có và mọi số liệu đã trích (RINSE `0,837`, autorater `93%`) áp dụng trực tiếp. Đây là chỗ rủi ro ngôn ngữ của bản kế hoạch trước **đóng lại**.

3.8 H Đo lường theo từng bước

```
đếm token theo TỪNG BƯỚC:  A0 * A1 * A2 * A3 * A4   và   C0 * C1 * C3 * C4 * C6
KHÔNG đếm tổng
```

Không tách theo bước thì khi chi phí lệch sẽ không biết lệch ở đâu.

3.9 J Chia tập — việc MỚI, do hai quyết định sinh ra

Ta đào nhãn trên **chính tập train của G-reasoner**. Hai hệ quả phải xử lý ngay từ đầu.

Rò rỉ nhãn vàng vào nhánh GFM.

```
loss giám sát của G-reasoner  =  trên "labeled relevant nodes"
"labeled relevant nodes"      =  chính supporting_documents
supporting_documents          =  chính chuẩn ngoài ta định dùng ở GĐ 1

=>  nhánh GFM (15/40) được TRAIN trên đúng nhãn vàng ta sắp đo bằng
=>  T trông đẹp giả, và ô "lỗi đồ thị" trông rỗng giả
```

Xử lý:

```
GĐ 1 hiệu chuẩn   PHẢI chạy trên split mà checkpoint CHƯA train
                  paper đánh giá trên 1.000 mẫu mỗi bộ - dùng đúng split đó
GĐ 2 đào nhãn     vẫn trên train set, như đã chốt
báo cáo           chỉ số chuẩn-ngoài TÁCH THEO NHÁNH NGUỒN:
                      pool chỉ-BM25   độc lập với nhãn vàng
                      pool đầy đủ      không độc lập
                  chênh lệch giữa hai con số CHÍNH LÀ cỡ của rò rỉ
```

Giữ held-out cho tầng B.

```
277.839 truy vấn có sẵn
cần      18.200 cho pha 1  +  200.000 cho pha 2  =  218.200   (79%)
còn      ~60.000
```

Phải khóa một phần làm held-out **trước khi** `GĐ 2` chạy. Không khóa thì tầng B train-on-train và mọi con số của nó phồng giả.

Đầu ra giai đoạn 0: chạy được một truy vấn end-to-end, biết nó tốn bao nhiêu token ở mỗi bước, và đã khóa split.

---

4. Giai đoạn 0.5 — Chạy thử cả bốn cổng trên test index · $945

Test set có index sẵn và `~3.000` truy vấn. Không đủ cho pha 2, nhưng **đủ để chạy toàn bộ bốn cổng **mà** không tốn một đồng indexing nào**.

```
chia 3.000 truy vấn test thành ba phần RỜI NHAU:
    500    hiệu chuẩn chuẩn ngoài        ->  CỔNG 1
  2.000    pha 1 thu nhỏ, train g_theta      ->  CỔNG 2
    500    cổng p                        ->  CỔNG 3

chi phí   500 x $0,37  +  2.000 x $0,38   =   $945
indexing  $0
```

Mục đích không phải sinh ra `g_` sản xuất. `N = 2.000` là thiếu so với `5.000-10.000` mà `g_` cần. Mục đích là biết bốn con số **trước khi** tiêu tiền indexing:

```
adq sai bao nhiêu                 CỔNG 1
Set Transformer có vượt GBM       CỔNG 2
p ở đâu, và tụt theo hop không    CỔNG 3
cỡ rò rỉ supporting_documents     1.3 - chênh pool chỉ-BM25 vs pool đầy đủ
```

`p = 0,45` ở đây thì ta biết thiết kế không chạy — với `$945` thay vì `$14.000`.

Hai lệch phải ghi rõ khi đọc kết quả, và chúng ngược chiều:

```
N_1 = 2.000 thiếu dữ liệu   ->  p THẤP hơn thực tế
test set sạch hơn train     ->  p CAO hơn thực tế
```

Nên con số này là **định hướng**, không phải con số nghiệm thu. `CỔNG 3` thật phải chạy lại sau khi có `N` đủ ở `GĐ 2`.

4.1 CỔNG 0.5 — có đáng index train không

```
p >= 0,80  và  CỔNG 1, CỔNG 2 đều qua     ->  index phần pha 1 ($820), chạy GĐ 2
p ~ 0,50-0,80                             ->  sửa trước, chạy lại GĐ 0.5. Vẫn $0 indexing.
p < 0,50                                  ->  DỪNG. Đã tiêu $1.130, chưa tiêu $13.000.
adq không đáng tin                        ->  sửa NLI/phân rã, chạy lại. Vẫn $0 indexing.
```

Đây là cổng rẻ nhất trong cả kế hoạch, và nó chặn **dòng chi phí lớn nhất**.

---

5. Giai đoạn 1 — Hiệu chuẩn chuẩn ngoài · $185

Mọi phép kiểm của tầng A đều **nội bộ**: `v(T)` `_q` do chính `v` đo, `_q` do chính `v_ref` đặt, `adq` do chính model NLI của ta phán. HotpotQA là chuẩn ngoài duy nhất sẵn có — nó gán sẵn **2 đoạn chống đỡ** và tối đa **5 câu chống đỡ** cho mỗi câu hỏi.

Không cần lớp khớp chunk. Bản kế hoạch trước có một bước *"khớp chunk của ta với đoạn của họ". *Hai quyết định ở mục* Đã chốt* xóa nó:

```
T của ta                =  tập TÊN TÀI LIỆU (node document layer)
supporting_documents    =  tập TÊN TÀI LIỆU trong train.json của framework
->  CÙNG không gian tên.  So sánh trực tiếp bằng phép so tập.
```

```
1.1  Chọn split           TEST SET - và nó khớp hoàn hảo hai điều kiện cùng lúc:
                          * là split checkpoint CHƯA train (chống rò rỉ, xem 0.J)
                          * CÓ index dựng sẵn  ->  $0 indexing
                          chạy trên TRAIN split thì cả ba con số dưới đây PHỒNG GIẢ

1.2  Chạy tầng A đầy đủ   500 truy vấn        500 x $0,37 = $185
                          (tính lại sau 0.B nếu độ dài đoạn khác giả định)

1.3  Đối chiếu T với supporting_documents - theo BỐN CA, không phải một con số
```

Vì sao không dùng precision/recall. `supporting_documents` là *"những đoạn một người đã dùng để trả lời", *không phải* "tập đủ tối tiểu"* theo định nghĩa §3.2. HotpotQA dựng theo lối 2-hop nên `|supporting_documents| = 2` **theo cấu trúc**, không theo phép đo. Nên nó chưa chắc tối tiểu, và precision/recall sẽ **trừ điểm ca ta làm tốt hơn nhãn**.

```
T c= support,  |T| < |support|    ta TỐI TIỂU HƠN annotation   ->  KHÔNG phải lỗi
T = support                       khớp
T  support                       phình
T có đoạn NGOÀI support           MỖI THAM SỐ  ->  đây mới là lỗi đang hunt
```

Và `adq` đáng tin không thì đo bằng: trong các ca `T` có đoạn ngoài `support`, `adq` có loại đúng chúng không.

Mỗi con số báo cáo HAI LẦN:

```
pool chỉ-BM25   độc lập với nhãn vàng
pool đầy đủ     nhánh GFM đã thấy nhãn vàng (xem 0.J)
chênh lệch giữa hai lần CHÍNH LÀ cỡ của rò rỉ
```

Con số (**b**) quan trọng nhất. Đem adq đi loại nhãn trên 200.000 truy vấn — và cho nó quyền bỏ ~45% truy vấn ở cửa C0 — mà **chưa bao giờ biết nó sai bao nhiêu** thì chỉ là đổi một lỗi không biết sang một lỗi khác không biết.

Hai lỗi cụ thể phải đo tại đây:

```
phân rã sót một hop     ->  bảng phủ thiếu cột  ->  T được nhận khi còn thiếu bằng chứng
NLI "neutral" giả       ->  thoát sớm oan, hoặc loại oan T tốt
```

5.1 CỔNG 1 — adq thành gì

```
mỗi tham số  < 5%        adq chỉ GHI làm metadata, không loại
             5-20%       adq GATE ở C6, giữ ca bị loại trong log train
             > 20%       adq vào CẢ hàm gain của greedy ở pha 1, không chỉ phép kiểm cuối

adq sai nhiều            CHƯA DÙNG ĐƯỢC
                         -> sửa phân rã, hoặc đổi model NLI, rồi chạy lại GĐ 1
                         -> ĐỪNG tiêu tiền pha 1
```

---

6. Giai đoạn 2 — Pha 1, đào nhãn · ~$3.850

```
2.0  INDEX phần pha 1     ~195.000 document  ->  OpenIE bằng LLM  ->  ~$820
                          đây cũng là phép thử cho chính bước indexing:
                          biết thật tốn bao nhiêu mỗi document TRƯỚC khi
                          cam kết 2,14 triệu document ở GĐ 5
                            không chạy nổi trong một session Colab nếu quy mô lớn -
                            cần xử lý theo lô CÓ THỂ TIẾP TỤC, checkpoint theo doc

2.1  Chạy A0 trên ~18.200 truy vấn
         chẻ câu hỏi + bảng phủ, 400 token mỗi truy vấn
         ĐO tỷ lệ thoát THẬT        (dự đoán 45%, suy từ MuSiQue 46,2% / HotpotQA 44,6%)

2.2  Chạy A1-A4 trên ~10.000 truy vấn sống sót
         A1  vòng 0        44.500 token
         A2  vòng 1        27.000     greedy có ràng buộc phủ
         A3  vòng 2       ~37.000     active learning quanh tau_q
         A4  vòng 3       ~15.000     xác nhận bằng model thật

2.3  Dựng log train       ~1.110.000 mẫu (q, S, v(S)) + tau_q, vòng, chọn/loại, |S|, cov
```

Nếu tỷ lệ thoát ở 2.1 khác xa 45% thì **sửa N nạp vào ngay tại đây**, đừng chờ hết giai đoạn. Thoát nhiều hơn dự thì nạp thêm; thoát ít hơn thì nạp ít đi.

Ca thoát ở A0 **không xóa** — chúng là cờ "pool thiếu", nguồn nuôi ô «lỗi đồ thị» §6.1 và vòng lặp ngược §6.6.

---

7. Giai đoạn 3 — Surrogate · 0 lượt LLM

```
3.1  GBM trên đặc trưng tay - MỐC PHẢI VƯỢT
         |S| * tổng_{kthuộcS} s^(q,k) * max s^ * số cụm S chạm
```

```
        * tương đồng cặp lớn nhất trong S * tương đồng với q * tổng v~_k khi có

3.2  Sàn tuyệt đối - model CHỈ nhìn |S|
        không vượt sàn này thì g_theta vô dụng, dù AUC gộp bao nhiêu

3.3  Set Transformer có mặt nạ
        đầu vào   q * S * s^(q,k) * v~_k * v(rỗng)        KHÔNG có tau_q
        che v~_k THEO TỪNG CHUNK, trộn ba khuôn theo tỷ lệ thật:
            pha 1 + cứu            cả 40 chunk có v~_k
            verification pha 2     CHỈ |T| chunk có
            tìm theo tầng ở C5     không chunk nào có
        loss  L_cls + lambda*L_rank,  L_rank so theo cặp TRONG một truy vấn

3.4  Chỉ số nghiệm thu
        AUC TRONG từng truy vấn, rồi trung bình qua các truy vấn
        KHÔNG dùng AUC gộp - tập lớn thì v cao nên số đẹp sẵn
        cân trọng số lớp: nhãn dương chỉ ~10-15%
```

Chú ý **rò rỉ ngữ cảnh pool**: mã hóa S cùng scalar theo từng chunk, **không** cho mô hình nhìn phần bù của S trong pool.

7.1 CỔNG 2 — kiến trúc

```
Set Transformer VƯỢT GBM trên AUC-trong-truy-vấn   ->  dùng Set Transformer
không vượt                                          ->  dùng GBM, đừng cố
cả hai không vượt sàn |S|                           ->  DỪNG, đặc trưng hoặc nhãn có vấn đề
```

---

8. Giai đoạn 4 — Cổng p · miễn phí

```
4.1  Giữ 500 truy vấn pha 1          nhãn đã có, không tốn lượt LLM nào
4.2  Chạy TOÀN BỘ đường pha 2        C0 -> C7, như thật
4.3  So T thu được với T thật đã biết
```

8.1 CỔNG 3 — có triển khai 200k không

```
p >= 0,90                  triển khai
p ~ 0,75                   tiền cứu ăn gần hết phần tiết kiệm
                           -> TĂNG N_1 trước, chưa tăng N_2
p tụt mạnh theo số hop     g_theta yếu ở ca bắc cầu
                           -> đổ thêm truy vấn multi-hop vào pha 1
p cao mà |T| phình         phần dư ÂM (trùng lặp) chưa học được
                           -> sửa kiến trúc, KHÔNG mua thêm nhãn
```

Đọc p **phân theo số hop**, không chỉ con số tổng. p tổng đẹp mà p ở 3-hop thấp thì nhãn sinh ra sẽ lệch về ca dễ — đúng thứ cả dự án muốn tránh.

---

9. Giai đoạn 5 — Pha 2, sản xuất theo lô · ~$8.500

```
5.0  INDEX phần pha 2     ~2,14 triệu document  ->  ~$9.000
                          CHỈ chạy sau khi CỔNG 3 qua trên N_1 đủ
                          đây là dòng chi phí LỚN NHẤT của cả chương trình
                          cần job chạy nhiều ngày, có thể tiếp tục

10 lô x 20.000 truy vấn

mỗi lô b:
    chạy C0 -> C7 bằng g_theta^(b)
    +- qua    ->  xuất T + DẤU XUẤT XỨ
    +- trượt  ->  hàng cứu
    lấy mẫu hàng cứu theo tầng  ->  chạy A1-A4 trên mẫu đó
    gộp mọi nhãn mới  ->  train g_theta^(b+1)
    đo p của lô này  ->  CỔNG 4
```

Dấu xuất xứ, bắt buộc trên mỗi nhãn:

```
T
+ phiên bản g_theta        g_theta^(1) ... g_theta^(10)    lô nào gán
+ đường đi              qua trực tiếp * giải trong dàn * tỉa * thử lại #k * được cứu
+ trọng số importance  N_s / n_s của tầng lấy mẫu
+ v(T) - tau_q           khoảng vượt ngưỡng
+ cov(T), J             phủ mấy mảnh trên mấy mảnh
```

Không có mấy trường này thì tầng B train trên một tập chất lượng không đều mà **không có cách nào biết**, và không lọc về được tập tin cậy nhất (`"chỉ pha 1 + ca được cứu"`).

Tỷ lệ cứu suy giảm, sàn 5%:

```
lô 1   ~40%        lô 5   ~25%        lô 10   ~10%        SÀN 5%, không bao giờ 0
```

Sàn phải giữ vì nếu không thì ô «lỗi đồ thị» mất nguồn và ta không còn đo được tỷ lệ trượt thật.

Trọng số importance — chỗ dễ dùng sai:

```
ĐO     p, tỷ lệ trượt, mọi đại lượng quần thể   ->  PHẢI dùng trọng số
TRAIN  g_theta                                      ->  ĐỪNG hạ trọng số ca cứu
                                                    chúng là hard example, muốn chúng NẶNG
```

Dùng chung một bộ trọng số cho cả hai là lỗi âm thầm: model vẫn train, chỉ là nó học nhẹ đi đúng ở chỗ nó yếu.

9.1 CỔNG 4 — ở ranh giới MỖI lô

```
p tăng đều qua các lô          chạy tiếp
p phẳng mà trượt vẫn nhiều     KHÔNG phải thiếu dữ liệu -> SỨC BIỂU ĐẠT của g_theta
                               dừng, sửa kiến trúc, đừng mua thêm nhãn
|T| phình đều                  dừng, sửa phần dư âm trước
p = 0,45 sau lô 1              DỪNG.  mới tốn ~$1.000, chưa tốn $12.500
```

---

10. Giai đoạn 6 — Tầng B

Tiêu thụ `(q, T)` và `V_neg`. `V_neg` lấy từ **pha 1**, không lấy pha 2 — ở pha 2 verification chỉ kiểm `T`, không kiểm cả họ `W`, nên `_i < 0` là bằng chứng yếu hơn hẳn.

Từ đây trở đi là §4 – §6 của tài liệu chính.

---

11. Ngân sách

Chia theo **cổng**, vì thứ tự chi tiêu mới là điểm chính.

```
                                    chi phí    tích lũy   cổng chặn sau đó
GĐ 0    hạ tầng, index test có sẵn      $0          $0
GĐ 0.5  chạy thử bốn cổng trên test    $945        $945    CỔNG 0.5  <- chặn $13.000
GĐ 1    hiệu chuẩn chuẩn ngoài         $185      $1.130    CỔNG 1
GĐ 2.0  index phần pha 1               $820      $1.950
GĐ 2    pha 1 đào nhãn               $3.850      $5.800
GĐ 3    surrogate                       $0      $5.800    CỔNG 2
GĐ 4    cổng p trên N_1 đủ               $0      $5.800    CỔNG 3    <- chặn $17.500
GĐ 5.0  index phần pha 2               $9.000    $14.800
GĐ 5    pha 2 + cứu                    $8.500    $23.300    CỔNG 4 x 10
                                      --------
                                      $23.300              ~104k nhãn đáng tin

chạy pipeline đầy đủ cho cả 200k      $76.000              -> 3,3 lần rẻ hơn
```

Bảng cũ ghi `$12.535` vì nó giả định ngầm rằng đồ thị có sẵn. Không có — chỉ test set có. `$9.820` indexing là phần thiếu.

Nhưng cấu trúc cổng vẫn giữ được điều quan trọng:

```
tiêu trước CỔNG 0.5    $945      -> nếu thiết kế sai, dừng ở đây
tiêu trước CỔNG 3    $5.800      -> nếu p thấp, không bao giờ trả $17.500 còn lại
```

Tức `75%` chi phí nằm **sau** cổng cuối cùng còn miễn phí.

---

12. Phải chốt trước khi giai đoạn 0 chạy

Năm quyết định của bản trước, nay còn hai.

```
ĐÃ CHỐT
#1  retriever    G-reasoner checkpoint pretrain, KHÔNG chạy sft_training
#4  corpus       train set HotpotQA + MuSiQue + 2Wiki, có đáp án vàng
    ngôn ngữ     tiếng Anh  ->  RỦI RO NGÔN NGỮ ĐÓNG
    chunk        một entry documents.json = một đoạn Wikipedia
    index        CÓ SẴN cho test set  *  KHÔNG có cho train set

CÒN TREO
#2  model judge   cái nào - nó quyết cost(S) và toàn bộ bảng chi phí
#3  model NLI     cái nào (giờ chọn dễ hơn hẳn, vì corpus là tiếng Anh)
#5  ba trích dẫn  Deegan-Packel 1978 * Banzhaf 1965 * Holler-Packel 1983
```

```
#6  model OpenIE  CHỈ cần khi index train. Model rẻ bớt 5-10 lần nhưng triple
                  xấu hơn -> nhánh GFM yếu thêm, mà ta vốn đã cố ý dùng
                  checkpoint chưa fine-tune
#7  index mấy bộ  cả ba, hay chỉ một? Chỉ HotpotQA thì rẻ ~3 lần nhưng mất ca
                  3-4 hop của MuSiQue - đúng ca cả dự án nhắm
#8  §3.8 parsimony  summary ở lại pool -> một summary phủ nhiều mảnh thì
                  parsimony -|S| cho nó THẮNG tập document tương đương.
                  Đếm SỐ NGUỒN BỊ BAO, hay chấp nhận T chỉ gồm summary + gắn cờ?
```

Và bốn câu hỏi hiện vật ở `0.A` phải trả lời trước khi viết dòng code nào:

```
đồ thị có được phát hành sẵn, hay phải tự dựng trên 2.972.931 document?
ba constructor - dùng cái nào?
một đồ thị cho cả ba bộ, hay một mỗi bộ?
split nào là split checkpoint chưa train?
```

Câu đầu quyết định `GĐ 0` có một job dựng đồ thị quy mô lớn hay không — chênh lệch công sức lớn nhất còn lại trong kế hoạch.

---

13. Rủi ro và dấu hiệu phát hiện

rủi ro	phát hiện bằng	giai đoạn
index train KHÔNG phát hành	0.A - đã xác nhận: chỉ test có	
chi phí OpenIE mỗi doc lệch giả định	GĐ 2.0 trên 195K doc đo thật	
job index 2,14M doc quá dài cho Colab	cần xử lý theo lô có thể tiếp tục	
ba constructor -> ba cách cắt khác nhau	0.A - chốt MỘT	
đồ thị trộn ba bộ	0.A - pool sẽ đầy đoạn bộ khác	
chunk có chồng lấn	0.B - nếu có, khử trùng lặp trước pool	
250 token/đoạn là GIẢ ĐỊNH	0.B - đo thật, rồi tính lại mọi bảng	
prompt caching không ăn	0.E - đếm token thật	
LLM chẻ sót hop	0.F - đối chiếu phân rã MuSiQue	
rò rỉ supporting_documents vào GFM	0.J + 1.3 - chênh BM25-only vs đầy đủ	
train-on-train khi đánh giá tầng B	0.J - khóa held-out TRƯỚC GĐ 2	
adq loại oan (neutral giả)	CỔNG 1, con số (b)	1.3
"pool thiếu" cao hơn 45%	2.1 - và nó làm ngân sách pha 2 đội	
g_theta không vượt GBM	CỔNG 2	3
g_theta không vượt sàn |S|	CỔNG 2 - nặng hơn, dừng hẳn	3
p thấp, hoặc tụt theo số hop	CỔNG 3	4
sức biểu đạt chứ không thiếu nhãn	CỔNG 4 - p phẳng qua các lô	5
nhãn không đồng chất qua các lô	dấu xuất xứ -> lọc được	5

Hai rủi ro của bản trước đã **đóng**: cách cắt chunk không khớp ba nơi (giờ pool/BM25/đồ thị dùng chung một tập node, đúng theo cấu trúc), và model NLI không phủ ngôn ngữ corpus (corpus là tiếng Anh).

---

14. Những con số chưa ai đo

Toàn bộ bảng ngân sách treo vào bốn con số sau, và mỗi cổng đo đúng một trong chúng:

```
$0,0042 mỗi document      GIẢ ĐỊNH          ->  0.A ước, GĐ 2.0 đo THẬT
                          chi phối $9.820, tức 42% ngân sách
```

```
250 token mỗi đoạn        GIẢ ĐỊNH          ->  0.B đo.  SAI chỗ này thì mọi bảng sai
p = 0,75                  GIẢ ĐỊNH          ->  CỔNG 0.5 đo rẻ, CỔNG 3 đo thật
30% mỗi lần thử lại       GIẢ ĐỊNH          ->  GĐ 5, lô 1 đo
45% thoát ở A0/C0         suy từ MuSiQue    ->  GĐ 2.1 đo
                          và chắc chắn LẠC QUAN, vì ta cố ý dùng
                          checkpoint chưa fine-tune
adq sai bao nhiêu         CHƯA BIẾT         ->  CỔNG 1 đo
cỡ rò rỉ nhãn vàng        CHƯA BIẾT         ->  1.3, chênh BM25-only vs đầy đủ
|T| = 2-3 là cột chính    theo lý thuyết    ->  GĐ 1.3 đo
```

Dòng đầu là dòng rẻ nhất và nguy hiểm nhất: nó đo được trong `0.B` bằng vài dòng code, nhưng nếu sai thì cả `$12.535` sai theo và không cổng nào bắt được — vì mọi cổng đều so với chính công thức đó.

Không cổng nào được bỏ qua để ”chạy nhanh hơn” — chúng rẻ chính vì chúng nằm trước chi phí, và bỏ một cổng là dồn rủi ro xuống giai đoạn đắt hơn mười lần.